

Lambda-Ausdrücke

Kategorie	Beschreibung
Definition	Ein Lambda-Ausdruck ist eine anonyme Funktion, die als Argument an Methoden übergeben oder einer Variablen zugewiesen werden kann.
Syntax	<code>(Parameter) -> { Ausdruck oder Block }</code>
Parameter	0, 1 oder mehrere Parameter möglich. Rundklammern sind optional, wenn nur ein Parameter vorhanden ist.
Rückgabebetyp	Wird aus dem Ausdruck abgeleitet, explizite Angabe nicht notwendig.
Typinferenz	Der Compiler kann den Typ der Parameter aus dem Kontext ableiten (Target Typing).
Funktionales Interface	Lambda-Ausdrücke erfordern ein funktionales Interface, also ein Interface mit genau einer abstrakten Methode.
Beispiel (einfache Form)	<code>x -> x * x</code> (nimmt <code>x</code> , gibt <code>x²</code> zurück)
Beispiel (mehrere Parameter)	<code>(a, b) -> a + b</code>
Beispiel (mehrzeiliger Block)	<code>(x, y) -> { int sum = x + y; return sum; }</code>
Beispiel mit Streams	<code>list.stream().map(x -> x * 2).collect(Collectors.toList())</code>
Beispiel mit Methodenreferenz	<code>list.forEach(System.out::println);</code>
Variable Capture	Zugriff auf Variablen aus dem umgebenden Kontext möglich, aber nur auf <code>final</code> oder <code>effectively final</code> Variablen.
Lexikalischer Scope	Lambda-Ausdrücke haben denselben Scope wie lokale Variablen und können nicht <code>this</code> zum äußeren Objekt referenzieren.
Laufzeit	Lambda-Ausdrücke werden zur Laufzeit als Instanzen eines funktionalen Interfaces kompiliert (synthetische Methoden in Bytecode).
Typkompatibilität	Kompatibel mit jedem funktionalen Interface mit passender Methodensignatur.
Vorteile	Kürzere, lesbarere und wartbare Code-Strukturen. Vermeidung von anonymen Klassen. Effiziente Parallelverarbeitung mit Streams.
Einschränkungen	Können nicht direkt Exception-Typen angeben (müssen in Try-Catch gekapselt oder eine Methode mit <code>throws</code> referenzieren).

Kategorie	Beschreibung
Performance	Lambda-Ausdrücke können durch <code>invokedynamic</code> optimiert werden, reduzieren Boilerplate-Code und verbessern die Parallelisierbarkeit.